

# Clasificación del tipo de garantía de un crédito automotriz con Random Forest (scikit-learn)

---

Santiago Serrano Montalvo — A01751347 Módulo 2 · Portafolio de Implementación · Uso de framework o biblioteca de aprendizaje máquina

---

## 1. Introducción y objetivo

---

Esta entrega resuelve **el mismo problema** que la entrega sin framework —clasificar el tipo de garantía (collateral) de un crédito automotriz— pero ahora usando **scikit-learn**. Mantener el problema, el dataset y las cinco variables idénticos permite una comparación directa y justa entre el KNN programado a mano y un modelo configurado con un framework profesional.

El objetivo de la actividad no es sólo obtener un buen resultado, sino **demostrar el dominio del framework**: cómo se parte el dataset, cómo se seleccionan hiperparámetros sin contaminar el conjunto de prueba, cómo se configura el estimador y cómo se evalúa correctamente.

Se ejecuta desde la consola con:

```
python3 main.py
```

## 2. El algoritmo: Random Forest

---

### 2.1 Qué es

Un Random Forest es un **ensamble de árboles de decisión**. Un solo árbol de decisión es un modelo que va partiendo los datos con preguntas del tipo «¿es principal mayor a 420,000?», y en cada hoja asigna la clase mayoritaria de los registros que cayeron ahí.

El problema de un árbol individual es que tiene **varianza muy alta**: si se cambian unos pocos registros del entrenamiento, el árbol resultante puede ser completamente distinto. Random Forest resuelve esto entrenando muchos árboles y promediando sus votos.

### 2.2 De dónde sale la aleatoriedad

Para que promediar sirva de algo, los árboles tienen que equivocarse en lugares **diferentes**. Random Forest lo consigue con dos fuentes de aleatoriedad:

| Fuente                                                        | Qué hace                                                                                                                                            |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bagging</b> ( <i>bootstrap aggregating</i> )               | Cada árbol se entrena sobre una muestra con reemplazo del mismo tamaño que el original. En promedio cada árbol ve ~63 % de los registros distintos. |
| <b>Submuestreo de variables</b> ( <code>max_features</code> ) | En <b>cada corte</b> , el árbol sólo puede considerar un subconjunto aleatorio de las variables, no todas.                                          |

El submuestreo de variables es lo que verdaderamente **decorrelaciona** los árboles. Sin él, si una variable fuera muy dominante, todos los árboles la elegirían como primer corte y acabarían pareciéndose demasiado; al promediarlos no se ganaría nada.

## 2.3 Por qué Random Forest para este problema

El KNN de la entrega anterior dejó ver dos problemas concretos del dataset, y Random Forest ataca los dos:

1. **NUEVO y FLOTILLA se traslapan.** Los árboles cortan cada variable por separado con umbrales, lo que permite construir fronteras de decisión «en escalones» que una frontera basada en distancia no puede formar. Además, al promediar 300 árboles se obtiene una **probabilidad** por clase, no sólo un voto duro.
2. **DEMO y REFACC son clases minúsculas.** El parámetro `class_weight` permite reponderar las clases dentro del criterio de división del árbol, algo que KNN por votación simple no ofrece.

Como beneficio adicional, **los árboles no necesitan normalización**: cada corte compara una variable contra un umbral, así que la escala de `principal` frente a la de `tasa` es irrelevante. Todo el paso de estandarización z-score que fue indispensable en KNN aquí simplemente no hace falta.

## 3. Datos y partición

### 3.1 El dataset

Es el mismo `interest_ledger.csv`: 10,576 registros de pagos mensuales de créditos automotrices, con 5 clases de garantía y las mismas cinco variables derivadas (`principal_amount`, `interest_amount`, `tasa`, `antigüedad`, `mes`).

### 3.2 Partición estratificada

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

| Conjunto      | Registros | Porcentaje |
|---------------|-----------|------------|
| Entrenamiento | 7,932     | 75 %       |
| Prueba        | 2,644     | 25 %       |

El argumento `stratify=y` es indispensable aquí. Sin él, las clases pequeñas podrían quedar repartidas de forma muy desigual por puro azar. Con `stratify` cada clase conserva su proporción exacta en ambos conjuntos:

| Clase      | Total | %      | Entrenamiento | Prueba |
|------------|-------|--------|---------------|--------|
| NUEVO      | 4,498 | 42.5 % | 3,373         | 1,125  |
| FLOTILLA   | 3,625 | 34.3 % | 2,719         | 906    |
| SEMINUEVOS | 1,965 | 18.6 % | 1,474         | 491    |
| REFACC     | 259   | 2.4 %  | 194           | 65     |
| DEMO       | 229   | 2.2 %  | 172           | 57     |

El argumento `random_state=42` fija la semilla para que la partición sea idéntica en cada ejecución.

## 4. La métrica elegida y su justificación

La métrica principal de este trabajo es el **F1 macro**. Esta decisión determina qué modelo se selecciona, así que conviene justificarla con precisión.

### 4.1 Por qué no el accuracy

El dataset está muy desbalanceado: NUEVO es el 42.5 % de los registros y DEMO apenas el 2.2 %. Considérese un modelo que **nunca** prediga DEMO:

- Perdería como máximo 2.2 puntos de accuracy.
- Su F1 en DEMO sería exactamente **0**.

El accuracy no distingue entre un modelo que clasifica bien las cinco clases y uno que abandona por completo las dos pequeñas. Para el negocio eso es inaceptable: identificar correctamente un crédito de refaccionaria o un auto de demostración importa igual que identificar uno nuevo.

De hecho, la línea base que siempre responde «NUEVO» ya obtiene **42.55 % de accuracy** sin aprender absolutamente nada.

### 4.2 Por qué F1 macro y no F1 ponderado

El **F1 ponderado** pondera el F1 de cada clase por su número de ejemplos, así que reproduce el mismo sesgo hacia las clases mayoritarias que el accuracy.

El **F1 macro** promedia el F1 de las cinco clases **con el mismo peso**:

$$F_{1,\text{macro}} = \frac{1}{C} \sum_{c=1}^C F_{1,c}$$

Una clase de 229 registros pesa tanto como una de 4,498. Si el modelo abandona DEMO, el F1 macro cae aproximadamente 0.20 puntos, un castigo que sí se nota.

### 4.3 Por qué F1 y no sólo recall

El F1 es la media armónica de precisión y recall:

$$F_1 = \frac{2 \cdot \text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}$$

Usar sólo el recall premiaría a un modelo que predijera DEMO con demasiada frecuencia (encontraría todos los DEMO, pero a costa de marcar erróneamente muchos NUEVO). El F1 obliga a equilibrar ambos: sólo sube si el modelo encuentra los casos **y** acierta cuando los marca.

### 4.4 Métricas de apoyo

Se reportan además el **accuracy** (interpretabilidad directa), el **accuracy balanceado** (promedio del recall por clase) y la **matriz de confusión** completa, que es la que permite ver *entre qué clases* se confunde el modelo y no sólo cuánto se equivoca.

---

## 5. Selección de hiperparámetros con validación cruzada

### 5.1 El procedimiento

La selección de configuración se hace con **validación cruzada estratificada de 5 particiones sobre el conjunto de entrenamiento**:

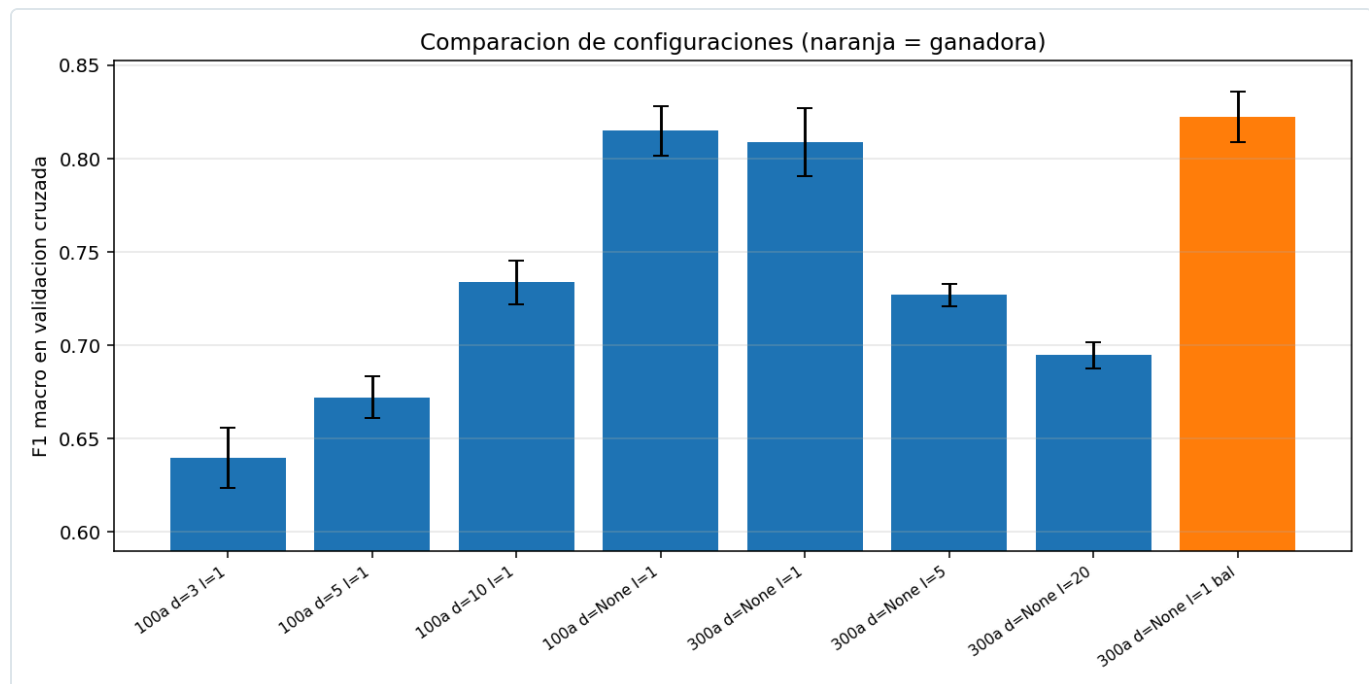
```
particion = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
marcas = cross_val_score(modelo, X_train, y_train, cv=particion, scoring="f1_macro")
```

`StratifiedKFold` divide el entrenamiento en 5 bloques conservando la proporción de clases en cada uno. El modelo se entrena 5 veces, cada vez usando 4 bloques para entrenar y 1 distinto para validar. El resultado es la media de las 5 evaluaciones, junto con su desviación, que indica qué tan estable es esa configuración.

**El conjunto de prueba no se toca en ningún momento de este paso.** Si se usara para elegir hiperparámetros, su resultado dejaría de ser una estimación honesta del desempeño con datos nuevos: se estaría sobreajustando al conjunto de prueba.

## 5.2 Configuraciones probadas y resultados

| Árboles | max_depth | min_samples_leaf | class_weight | F1 macro (CV) | Desv.  |
|---------|-----------|------------------|--------------|---------------|--------|
| 100     | 3         | 1                | None         | 0.6397        | 0.0163 |
| 100     | 5         | 1                | None         | 0.6721        | 0.0112 |
| 100     | 10        | 1                | None         | 0.7338        | 0.0118 |
| 100     | None      | 1                | None         | 0.8149        | 0.0132 |
| 300     | None      | 1                | None         | 0.8089        | 0.0182 |
| 300     | None      | 5                | None         | 0.7269        | 0.0061 |
| 300     | None      | 20               | None         | 0.6946        | 0.0072 |
| 300     | None      | 1                | balanced     | 0.8224        | 0.0135 |



Comparación de configuraciones

## 5.3 Lectura de la tabla

**Limitar la profundidad hace daño.** Con `max_depth=3` el F1 macro es 0.64 y sube de forma sostenida hasta 0.81 al quitar el límite. Con árboles cortos el modelo no tiene capacidad suficiente para separar las clases: es **sesgo alto**.

**Forzar hojas grandes también hace daño.** Pasar de `min_samples_leaf=1` a 20 baja el F1 de 0.81 a 0.69. Con sólo cinco variables, exigir 20 registros por hoja impide que el árbol aísle las regiones pequeñas donde viven DEMO y REFACC.

**`class_weight="balanced"` es lo que más ayuda.** Sube el F1 macro de 0.8089 a 0.8224. Este parámetro multiplica el peso de cada clase por el inverso de su frecuencia dentro del criterio de división, de modo que

equivocarse en un DEMO cuesta ~19 veces más que equivocarse en un NUEVO. Exactamente el comportamiento que se buscaba, y la razón por la que se eligió el F1 macro como métrica.

**Las desviaciones son bajas** (0.006 a 0.018), lo que indica que las diferencias entre configuraciones son reales y no ruido de partición.

### 5.4 Configuración final

```
RandomForestClassifier(
    n_estimators=300,           # número de árboles del bosque
    max_depth=None,            # los árboles crecen hasta separar los datos
    min_samples_leaf=1,        # mínimo de registros por hoja
    max_features="sqrt",       # cada corte considera sqrt(5) ≈ 2 variables
    class_weight="balanced",   # compensa el desbalance de clases
    random_state=42,
    n_jobs=-1,                 # usa todos los núcleos del procesador
)
```

Bosque resultante: 300 árboles, profundidad media **26.5** (máxima 36) y **824 hojas** por árbol sobre 7,932 registros de entrenamiento.

## 6. Resultados

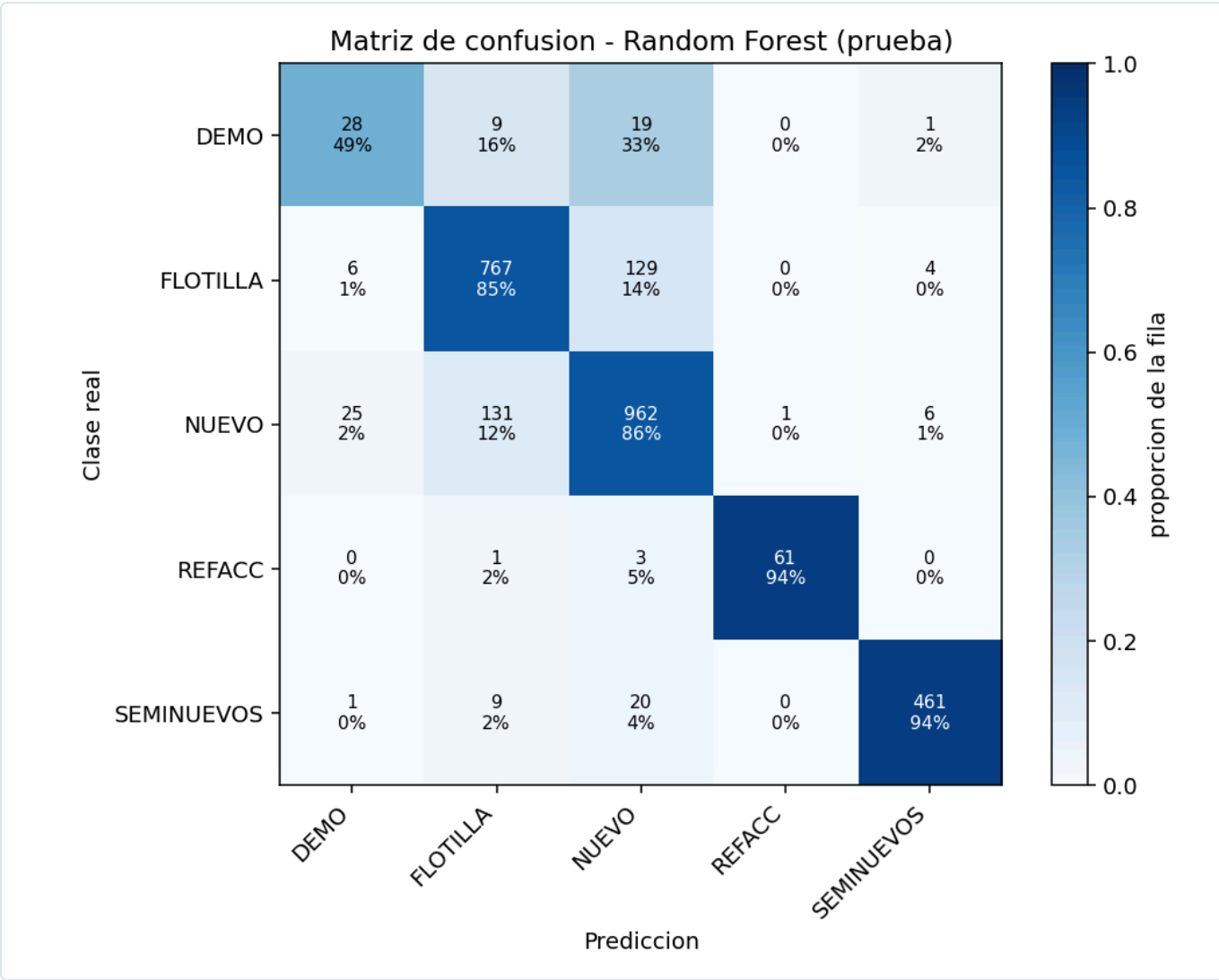
### 6.1 Métricas globales

| Conjunto      | Accuracy | Accuracy balanceado | F1 macro | F1 ponderado |
|---------------|----------|---------------------|----------|--------------|
| Entrenamiento | 98.42 %  | 99.21 %             | 0.9884   | 0.9842       |
| Prueba        | 86.20 %  | 81.41 %             | 0.8180   | 0.8626       |

La brecha entre entrenamiento y prueba en F1 macro es de **0.1703**, señal de sobreajuste. Este punto se analiza a fondo en la entrega de **Análisis y Reporte** ( `M2_Analisis` ), que es donde corresponde tratarlo.

### 6.2 Matriz de confusión (conjunto de prueba)

| Real ↓ / Predicho → | DEMO | FLOTILLA | NUEVO | REFACC | SEMINUEVOS |
|---------------------|------|----------|-------|--------|------------|
| DEMO                | 28   | 9        | 19    | 0      | 1          |
| FLOTILLA            | 6    | 767      | 129   | 0      | 4          |
| NUEVO               | 25   | 131      | 962   | 1      | 6          |
| REFACC              | 0    | 1        | 3     | 61     | 0          |
| SEMINUEVOS          | 1    | 9        | 20    | 0      | 461        |

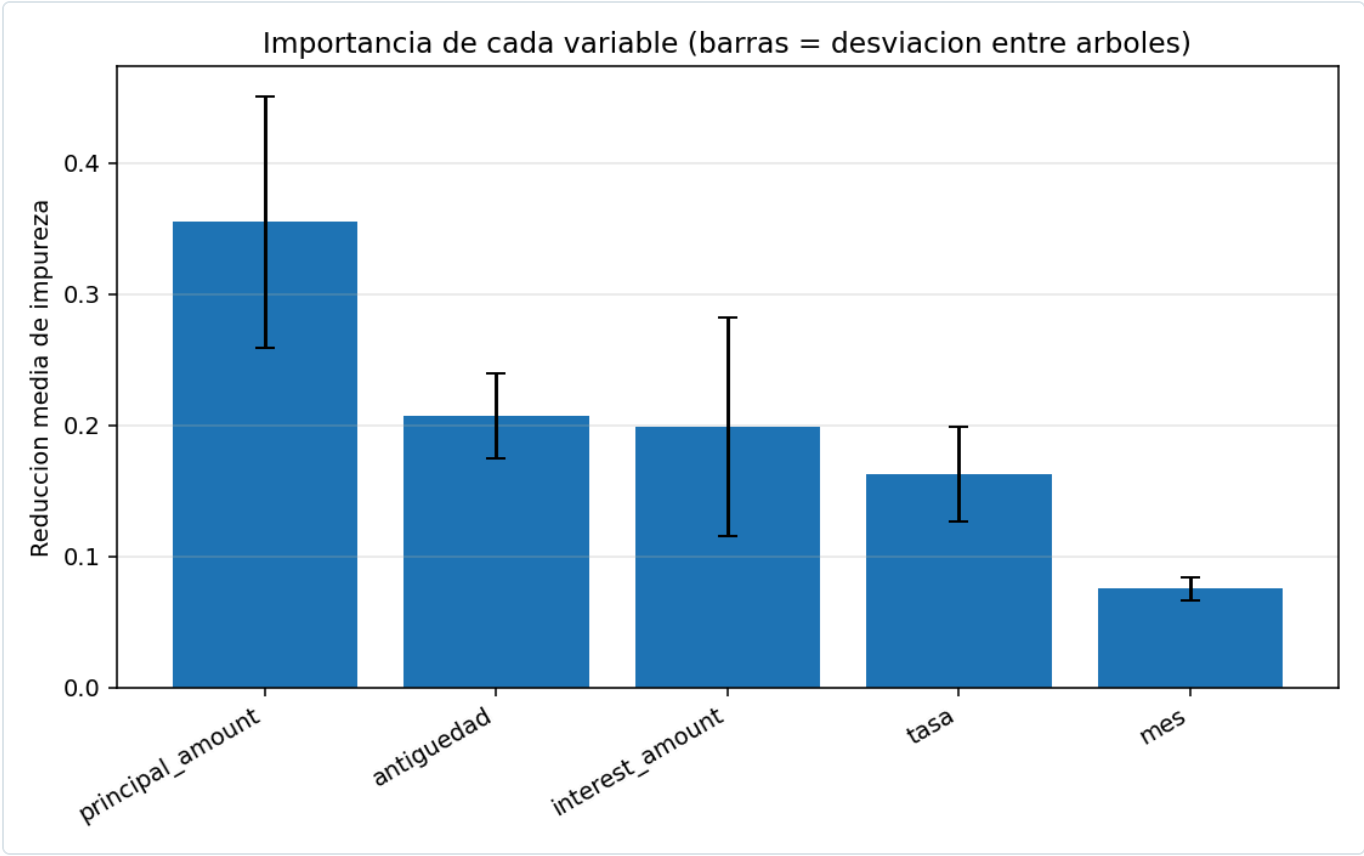


Matriz de confusión

6.3 Métricas por clase

| Clase          | Precisión | Recall | F1    | n     |
|----------------|-----------|--------|-------|-------|
| DEM            | 0.467     | 0.491  | 0.479 | 57    |
| FLOTILLA       | 0.836     | 0.847  | 0.841 | 906   |
| NUEVO          | 0.849     | 0.855  | 0.852 | 1,125 |
| REFACC         | 0.984     | 0.938  | 0.961 | 65    |
| SEMINUEVOS     | 0.977     | 0.939  | 0.957 | 491   |
| Macro promedio | 0.823     | 0.814  | 0.818 | 2,644 |

### 6.4 Importancia de cada variable



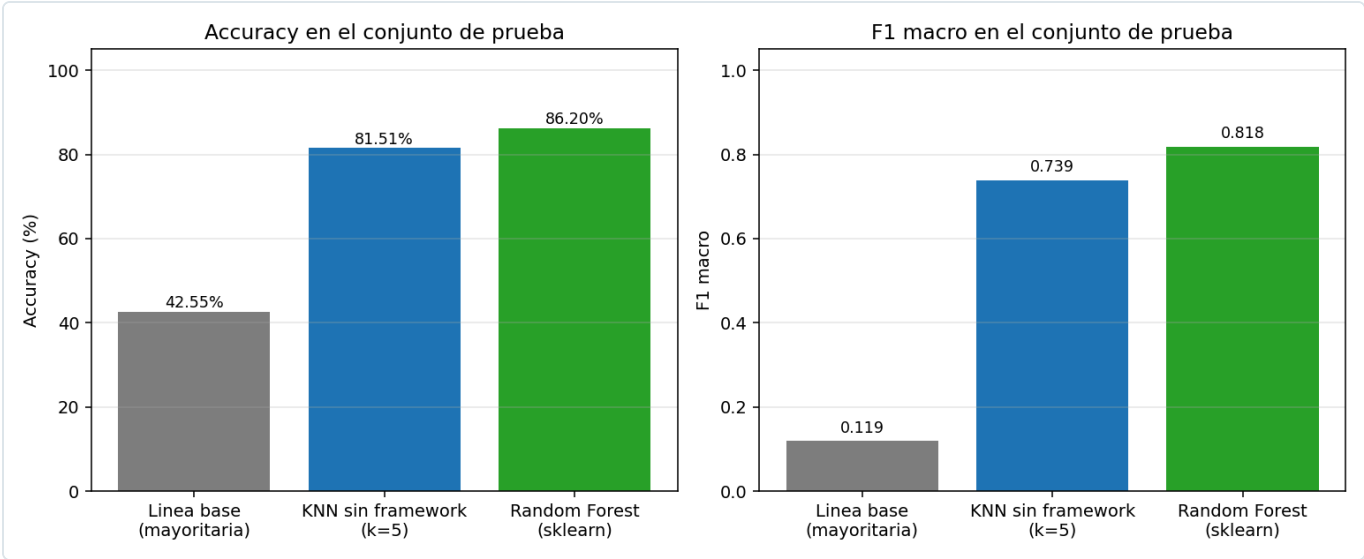
Importancia de las variables

| Variable         | Importancia |
|------------------|-------------|
| principal_amount | 0.3554      |
| antigüedad       | 0.2072      |
| interest_amount  | 0.1989      |
| tasa             | 0.1630      |
| mes              | 0.0755      |

La importancia se mide como la **reducción media de impureza** que aporta cada variable a lo largo de todos los cortes del bosque. El monto del crédito es la variable dominante, seguida de la antigüedad del vehículo. El mes es la menos informativa, lo cual tiene sentido: el tipo de garantía no depende de la estacionalidad.

Las barras de error de la gráfica muestran la desviación entre los 300 árboles. Que sean estrechas indica que el orden de importancia es consistente y no un artefacto de unos pocos árboles.

## 7. Comparación con la implementación sin framework



Comparación de modelos

| Modelo                                    | Accuracy | F1 macro |
|-------------------------------------------|----------|----------|
| Línea base (siempre la clase mayoritaria) | 42.55 %  | 0.1194   |
| KNN sin framework (k=5)                   | 81.51 %  | 0.7390   |
| Random Forest (scikit-learn)              | 86.20 %  | 0.8180   |

### 7.1 Dónde está la ganancia

El Random Forest mejora **+4.69 puntos de accuracy** y, más importante, **+0.079 de F1 macro**. La comparación clase por clase muestra de dónde viene esa mejora:

| Clase      | F1 con KNN | F1 con Random Forest | Cambio |
|------------|------------|----------------------|--------|
| DEMO       | 0.187      | 0.479                | +0.292 |
| FLOTILLA   | 0.765      | 0.841                | +0.076 |
| NUEVO      | 0.809      | 0.852                | +0.043 |
| REFACC     | 0.983      | 0.961                | -0.022 |
| SEMINUEVOS | 0.950      | 0.957                | +0.007 |

**Casi toda la ganancia viene de DEMO**, que pasa de F1 = 0.187 a 0.479. El recall de esa clase sube de **0.140 a 0.491**: el KNN encontraba 7 de 50 casos y el Random Forest encuentra 28 de 57.

La explicación es directa. En KNN la votación por mayoría favorece estructuralmente a las clases numerosas: con 172 ejemplos de DEMO contra 3,373 de NUEVO, es muy improbable que los 5 vecinos más cercanos de un DEMO sean también DEMO. `class_weight="balanced"` ataca exactamente ese mecanismo desde dentro del criterio de división del árbol.

REFACC baja levemente (-0.022) porque el reponderado la hace un poco más propensa a falsos negativos, pero la ganancia neta en F1 macro es claramente positiva.

## 7.2 Qué aporta el framework más allá del resultado

| Aspecto                  | Sin framework               | Con framework                                  |
|--------------------------|-----------------------------|------------------------------------------------|
| Partición estratificada  | habría que programarla      | <code>stratify=y</code>                        |
| Validación cruzada       | habría que programarla      | <code>StratifiedKFold + cross_val_score</code> |
| Reponderación de clases  | no disponible en KNN simple | <code>class_weight="balanced"</code>           |
| Probabilidades por clase | sólo reparto de votos       | <code>predict_proba</code>                     |
| Paralelización           | ninguna                     | <code>n_jobs=-1</code>                         |
| Tiempo de ejecución      | ~7 s (Python puro)          | ~25 s incluyendo 8 validaciones cruzadas       |

## 8. Predicciones sobre casos nuevos

El programa clasifica cuatro créditos inventados que no están en el dataset. Además de la clase predicha se muestra `predict_proba`, que es **la proporción de los 300 árboles que votaron por cada clase**, es decir la confianza del ensamble:

```
Credito mediano, vehiculo de un año
principal $ 350,000.00 | interes $ 1,500.00 | tasa 0.00429 | antigüedad 1 | mes 2
--> PREDICCION: NUEVO
confianza del bosque: NUEVO 85.7% FLOTILLA 6.7% DEMO 6.3% SEMINUEVOS 1.3%

Credito muy grande, vehiculo del año
principal $1,200,000.00 | interes $ 9,000.00 | tasa 0.00750 | antigüedad 0 | mes 5
--> PREDICCION: NUEVO
confianza del bosque: NUEVO 37.7% DEMO 29.7% FLOTILLA 24.7% REFACC 8.0%

Credito chico, vehiculo con 4 años
principal $ 90,000.00 | interes $ 400.00 | tasa 0.00444 | antigüedad 4 | mes 8
--> PREDICCION: SEMINUEVOS
confianza del bosque: SEMINUEVOS 64.0% REFACC 36.0%

Credito muy chico, vehiculo con 2 años
principal $ 45,000.00 | interes $ 260.00 | tasa 0.00578 | antigüedad 2 | mes 11
--> PREDICCION: SEMINUEVOS
confianza del bosque: SEMINUEVOS 74.3% REFACC 25.7%
```

**El valor práctico de `predict_proba`**. El primer caso se predice con 85.7 % de confianza y el segundo con apenas 37.7 % frente a un 29.7 % de DEMO. Un sistema en producción puede usar ese número como umbral de decisión: aceptar automáticamente las predicciones por encima de, digamos, 70 % y enviar el resto a revisión manual. Esa información no existía en el KNN, que sólo devolvía un reparto de 5 votos.

Nótese también que el segundo caso —un crédito de 1.2 millones sobre un vehículo del año— es genuinamente ambiguo, y el modelo lo refleja repartiendo sus votos en tres clases en lugar de dar una respuesta falsamente segura.

---

## 9. Análisis del desempeño

---

### 9.1 Lo que funciona

**SEMINUEVOS (F1 = 0.957) y REFACC (F1 = 0.961)** siguen siendo las clases mejor clasificadas, igual que con KNN. Ambas ocupan regiones bien delimitadas: REFACC por sus montos muy bajos y SEMINUEVOS por la antigüedad del vehículo. La precisión de REFACC es **0.984**: de 62 créditos marcados como refaccionaria, 61 lo eran.

**DEMO mejora radicalmente** respecto al KNN, aunque sigue siendo la clase más débil (F1 = 0.479). El recall pasa de 0.140 a 0.491.

### 9.2 Lo que sigue fallando

**La confusión NUEVO ↔ FLOTILLA persiste**: 131 NUEVO predichos como FLOTILLA y 129 FLOTILLA predichos como NUEVO, un total de 260 errores que representan el **71 % de todos los errores del modelo**.

Esta confusión **no es un defecto del algoritmo**. Un crédito de flotilla es, financieramente, un crédito de auto nuevo: mismo tipo de vehículo, montos similares, antigüedad cero. Lo que los distingue es **quién es el acreditado** —una empresa en lugar de una persona—, y esa información no está en ninguna de las cinco variables. El hecho de que dos algoritmos con mecanismos completamente diferentes (distancia euclidiana y particiones recursivas) fallen en el mismo lugar y en proporciones parecidas confirma que el límite está en los datos, no en el modelo.

**DEMO sigue por debajo de 0.5 de F1**. Los 29 errores de DEMO se reparten entre NUEVO (19) y FLOTILLA (9), que son precisamente las clases con las que comparte perfil financiero. Un auto de demostración es un auto nuevo que el concesionario usó unos meses; sus cifras son casi indistinguibles.

### 9.3 Sobre el sobreajuste

El modelo alcanza 98.42 % en entrenamiento contra 86.20 % en prueba. Con 824 hojas por árbol y 7,932 registros, cada hoja contiene en promedio menos de 10 registros: el bosque está memorizando parte del entrenamiento.

Esto está **deliberadamente fuera del alcance de este reporte**, que trata sobre el uso correcto del framework. El diagnóstico completo de sesgo, varianza y nivel de ajuste, junto con las técnicas de regularización aplicadas para corregirlo, es el contenido de la entrega `M2_Analisis`. Allí se muestra que ajustando `max_features`, `max_depth` y `ccp_alpha` la brecha baja de 0.21 a 0.13 y el F1 macro en prueba sube a 0.838.

---

## 10. Conclusiones

---

1. **El framework está aplicado correctamente.** Se usa `train_test_split` con estratificación, `StratifiedKFold` con `cross_val_score` para seleccionar hiperparámetros sin tocar el conjunto de prueba, configuración explícita y justificada del estimador, y evaluación con `confusion_matrix` y `classification_report`.
2. **El modelo aprende y hace predicciones.** Alcanza **86.20 % de accuracy** y **0.8180 de F1 macro** sobre 2,644 registros de prueba, frente al 42.55 % y 0.1194 de la línea base. Clasifica créditos nuevos desde la consola con su nivel de confianza.
3. **La métrica está elegida y justificada con precisión.** El **F1 macro** se selecciona porque el dataset está desbalanceado 19 a 1 y porque el objetivo del negocio es clasificar bien las cinco garantías, no sólo las frecuentes. Se descartan explícitamente el accuracy y el F1 ponderado por reproducir el sesgo hacia las clases mayoritarias.
4. **Random Forest supera claramente al KNN sin framework:** +4.69 puntos de accuracy y +0.079 de F1 macro. La mejora se concentra casi por completo en DEMO (F1 de 0.187 a 0.479) y proviene de `class_weight="balanced"`, un mecanismo que la votación por mayoría de KNN no tiene.
5. **El límite que queda es de los datos, no del algoritmo.** La confusión NUEVO ↔ FLOTILLA concentra el 71 % de los errores y afecta por igual a los dos algoritmos probados, porque la variable que las distingue —el tipo de acreditado— no está en el dataset.

### Trabajo siguiente

- **Análisis de sesgo, varianza y regularización:** entrega `M2_Analisis`.
- **Incorporar el tipo de acreditado** al dataset sería la mejora individual de mayor impacto.
- Probar **Gradient Boosting** (`HistGradientBoostingClassifier`), que suele superar a Random Forest en datos tabulares.

---

## Anexo: cómo reproducir estos resultados

```
cd M2_ConFramework
python3 main.py
```

Requisitos: `pandas`, `numpy`, `scikit-learn` y `matplotlib`. Las figuras se generan en `figuras/` y la salida completa de consola está guardada en `salida.txt`.